

# Documentação BEM WMS

## 1. Título do projeto

**BEM WMS — Warehouse Management System de Controle de Entrada e Saída de Estoque**

---

## 2. Nome dos integrantes

**Integrantes do grupo:**

- Emily Kristin Garcia
  - Bruno Batista Xavier
  - Maria Eduarda Martins de Souza
- 

## 3. Objetivo do projeto

O objetivo do projeto é desenvolver um sistema de controle de estoque para materiais escolares, permitindo o gerenciamento de entradas, saídas, lotes, status dos materiais e indicadores financeiros.

O **BEM WMS** foi criado para simular um sistema de armazém/estoque, no qual é possível cadastrar materiais escolares, consultar itens cadastrados, listar materiais ativos e inativos, editar registros, remover materiais e registrar saídas de estoque por lote.

Além das funcionalidades básicas de uma API, o sistema também possui uma interface web em modo dark, desenvolvida com HTML, CSS e JavaScript, permitindo que o usuário interaja com os dados de forma visual e intuitiva.

O sistema também apresenta um dashboard financeiro, exibindo informações como valor investido, venda estimada, lucro estimado e ROI, possibilitando uma visão mais completa sobre o estoque.

A proposta da equipe foi criar uma aplicação simples, funcional e organizada, utilizando **ASP.NET Core Minimal API** no back-end e um front-end consumindo os dados da API por meio de requisições HTTP.

---

## 4. Estrutura da solução

A entidade principal do sistema é o **MaterialEscolar**.

A classe `MaterialEscolar` possui os seguintes atributos:

| Atributo        | Tipo | Descrição                       |
|-----------------|------|---------------------------------|
| <code>Id</code> | int  | Identificador único do material |

| Atributo  | Tipo       | Descrição                                  |
|-----------|------------|--------------------------------------------|
| Nome      | string     | Nome do material escolar                   |
| Categoria | string     | Categoria do material                      |
| Ativo     | bool       | Indica se o material está ativo ou inativo |
| Lotes     | List<Lote> | Lista de lotes vinculados ao material      |

Além disso, existe a entidade **Lote**, que representa os lotes associados aos materiais escolares.

A classe `Lote` possui os seguintes atributos:

| Atributo       | Tipo      | Descrição                                  |
|----------------|-----------|--------------------------------------------|
| Id             | int       | Identificador único do lote                |
| Codigo         | string    | Código do lote                             |
| Quantidade     | int       | Quantidade de itens disponíveis no lote    |
| DataEntrada    | DateTime  | Data de entrada do lote no estoque         |
| DataVencimento | DateTime? | Data de vencimento do lote, quando existir |
| ValorCusto     | decimal   | Valor de custo unitário do item            |
| ValorVenda     | decimal   | Valor de venda unitário do item            |

Também existe a entidade **SaidaEstoque**, utilizada para registrar a saída de materiais.

A classe `SaidaEstoque` possui os seguintes atributos:

| Atributo   | Tipo | Descrição                                |
|------------|------|------------------------------------------|
| MaterialId | int  | Identificador do material que terá saída |
| Quantidade | int  | Quantidade que será retirada do estoque  |

A API se relaciona com os dados por meio de uma lista simulada em memória, localizada no arquivo `BancoSimulado.cs`. Essa abordagem foi utilizada para fins acadêmicos e demonstração do funcionamento da API, sem conexão direta com um banco de dados.

## 5. Funcionalidades do sistema

O **BEM WMS** possui as seguintes funcionalidades:

- Cadastro de materiais escolares;
- Cadastro de dados de lote;
- Consulta de todos os materiais;
- Consulta de materiais ativos;
- Consulta de materiais inativos;

- Consulta de material por ID;
- Consulta de lotes por material;
- Edição de materiais cadastrados;
- Remoção de materiais;
- Registro de saída de estoque;
- Controle de saída por lote;
- Dashboard financeiro;
- Cálculo de valor investido;
- Cálculo de venda estimada;
- Cálculo de lucro estimado;
- Cálculo de ROI;
- Tabela de itens, códigos e lotes;
- Interface web em modo dark;
- Consumo da API pelo front-end.

---

## 6. Regra de saída por lote

O sistema realiza a saída de estoque considerando os lotes disponíveis para cada material.

Quando o material possui data de vencimento, o sistema prioriza os lotes com vencimento mais próximo. Essa regra é conhecida como **FEFO**.

FEFO - First Expire, First Out

Ou seja:

Primeiro que vence, primeiro que sai.

Para materiais sem data de vencimento, o sistema utiliza a data de entrada como critério de ordenação, priorizando os lotes mais antigos.

---

## 7. Cálculos financeiros

O sistema possui um dashboard que apresenta informações financeiras do estoque.

### Valor investido

O valor investido representa o custo total dos itens ainda disponíveis em estoque.

$$\text{Valor Investido} = \text{Quantidade em Estoque} \times \text{Valor de Custo Unitário}$$

### Venda estimada

A venda estimada representa o valor que seria obtido caso todo o estoque atual fosse vendido.

$$\text{Venda Estimada} = \text{Quantidade em Estoque} \times \text{Valor de Venda Unitário}$$

### Lucro estimado

O lucro estimado representa a diferença entre a venda estimada e o valor investido.

$$\text{Lucro Estimado} = \text{Venda Estimada} - \text{Valor Investido}$$

### ROI

O ROI representa o retorno sobre o investimento em percentual.

$$\text{ROI (\%)} = (\text{Lucro Estimado} / \text{Valor Investido}) \times 100$$

Exemplo:

Quantidade: 50  
Custo unitário: R\$ 8,50  
Venda unitária: R\$ 14,90

Cálculo:

Valor Investido =  $50 \times 8,50 = \text{R\$ } 425,00$   
Venda Estimada =  $50 \times 14,90 = \text{R\$ } 745,00$   
Lucro Estimado =  $745,00 - 425,00 = \text{R\$ } 320,00$   
ROI =  $(320,00 / 425,00) \times 100 = 75,29\%$

## 8. Endpoints da API

| Método | Rota                      | Descrição                                                  |
|--------|---------------------------|------------------------------------------------------------|
| GET    | /                         | Retorna uma mensagem informando que a API está funcionando |
| GET    | /api/materiais            | Lista todos os materiais cadastrados                       |
| GET    | /api/materiais/{id}       | Busca um material específico pelo seu identificador        |
| GET    | /api/materiais/{id}/lotes | Lista os lotes vinculados a um material                    |
| GET    | /api/materiais/ativos     | Lista apenas os materiais ativos                           |
| GET    | /api/materiais/inativos   | Lista apenas os materiais inativos                         |

| Método | Rota                              | Descrição                                                    |
|--------|-----------------------------------|--------------------------------------------------------------|
| POST   | <code>/api/materiais</code>       | Cadastra um novo material                                    |
| PUT    | <code>/api/materiais/{id}</code>  | Edita um material cadastrado                                 |
| DELETE | <code>/api/materiais/{id}</code>  | Remove um material pelo seu identificador                    |
| POST   | <code>/api/materiais/saida</code> | Registra a saída de estoque de um material                   |
| GET    | <code>/api/dashboard</code>       | Retorna os indicadores financeiros e operacionais do sistema |

## 9. Detalhamento dos endpoints

### GET `/`

Endpoint utilizado para verificar se a API está funcionando corretamente.

**Retorno esperado:**

```
API de Materiais Escolares funcionando!
```

### GET `/api/materiais`

Retorna todos os materiais cadastrados na lista simulada.

A resposta inclui os dados do material e seus respectivos lotes, como quantidade, código do lote, data de entrada, data de vencimento, valor de custo e valor de venda.

### GET `/api/materiais/{id}`

Busca um material pelo seu `Id`.

Caso o material seja encontrado, a API retorna seus dados completos.

Caso contrário, retorna:

```
Material não encontrado.
```

### GET `/api/materiais/{id}/lotes`

Retorna todos os lotes vinculados a um material específico.

Essa rota permite visualizar os lotes cadastrados para determinado item.

---

#### **GET** /api/materiais/ativos

Retorna somente os materiais em que o campo **Ativo** está definido como **true**.

---

#### **GET** /api/materiais/inativos

Retorna somente os materiais em que o campo **Ativo** está definido como **false**.

---

#### **POST** /api/materiais

Permite cadastrar um novo material na lista simulada.

O corpo da requisição deve conter os dados do material e seu respectivo lote.

Exemplo:

```
{
  "id": 101,
  "nome": "Mochila Escolar",
  "categoria": "Mochila",
  "ativo": true,
  "lotes": [
    {
      "id": 101,
      "codigo": "MOC001",
      "quantidade": 20,
      "dataEntrada": "2026-04-10T00:00:00",
      "dataVencimento": null,
      "valorCusto": 45.90,
      "valorVenda": 89.90
    }
  ]
}
```

---

#### **PUT** /api/materiais/{id}

Permite editar um material já cadastrado.

A rota atualiza os dados do material, como nome, categoria, status e informações do lote.

Exemplo:

```
{
  "id": 101,
  "nome": "Mochila Escolar Reforçada",
  "categoria": "Mochila",
  "ativo": true,
  "lotes": [
    {
      "id": 101,
      "codigo": "MOC001",
      "quantidade": 20,
      "dataEntrada": "2026-04-10T00:00:00",
      "dataVencimento": null,
      "valorCusto": 45.90,
      "valorVenda": 99.90
    }
  ]
}
```

Caso o material exista, a API retorna uma mensagem de sucesso.

Caso contrário, retorna:

Material não encontrado.

---

## **DELETE** /api/materiais/{id}

Remove um material da lista com base no seu **Id**.

Caso o material exista, ele é removido e a API retorna:

Material removido com sucesso.

Caso não exista, retorna:

Material não encontrado.

---

## **POST** /api/materiais/saida

Registra a saída de estoque de um material.

O corpo da requisição deve conter o ID do material e a quantidade que será retirada.

Exemplo:

```
{
  "materialId": 1,
  "quantidade": 10
}
```

Caso exista estoque suficiente, a API atualiza a quantidade dos lotes e retorna uma mensagem de sucesso.

Caso não exista estoque suficiente, retorna:

Estoque insuficiente para realizar a saída.

---

## GET /api/dashboard

Retorna os indicadores operacionais e financeiros do sistema.

A resposta contém:

- Total de materiais;
- Total de ativos;
- Total de inativos;
- Total de unidades em estoque;
- Valor investido;
- Venda estimada;
- Lucro estimado;
- ROI percentual;
- Dados financeiros por material.

---

## 10. Front-end

Além da API, o projeto possui uma interface web desenvolvida com **HTML, CSS e JavaScript**.

A interface possui a nomenclatura:

BEM WMS  
Warehouse Management System de controle de entrada e saída de estoque

O front-end possui as seguintes seções:

- Dashboard;
- Cadastro de material;
- Registro de saída de estoque;
- Tabela de itens, códigos e lotes;
- Lista de materiais cadastrados.



---

## 11. Funcionalidades do front-end

### Dashboard

A seção de dashboard apresenta os principais indicadores do sistema:

- Total de materiais;
- Itens ativos;
- Itens inativos;
- Unidades em estoque;
- Valor investido;
- Venda estimada;
- Lucro estimado;
- ROI.

---

### Dashboard financeiro por material

Além dos cards principais, o sistema possui uma tabela financeira por material.

A tabela apresenta:

- Material;
- Categoria;
- Estoque;
- Valor investido;
- Venda estimada;
- Lucro;
- ROI.

---

### Cadastro de material

O formulário de cadastro permite inserir:

- ID do material;
  - Nome;
  - Categoria;
  - Status;
  - ID do lote;
  - Código do lote;
  - Quantidade em estoque;
  - Valor de custo unitário;
  - Valor de venda unitário;
  - Data de entrada;
  - Data de vencimento.
-

## Edição de material

Na listagem de materiais, o botão **Editar material** carrega os dados do item no formulário.

Após a edição, o botão passa a salvar as alterações utilizando a rota:

```
PUT /api/materiais/{id}
```

Também existe o botão **Cancelar edição**, que limpa o formulário e retorna ao modo de cadastro.

---

## Registro de saída

A seção de saída permite selecionar um material e informar a quantidade que será retirada do estoque.

Após registrar a saída, o sistema atualiza automaticamente:

- Quantidade em estoque;
  - Dashboard;
  - Tabela de itens;
  - Lista de materiais.
- 

## Tabela de itens, códigos e lotes

A tabela de itens apresenta uma consulta rápida dos dados cadastrados.

Ela exibe:

- ID do material;
  - Material;
  - Categoria;
  - Status;
  - ID do lote;
  - Código do lote;
  - Quantidade;
  - Data de entrada;
  - Data de vencimento;
  - Custo unitário;
  - Venda unitária.
- 

## Filtros e navegação

A interface possui botões de navegação e filtros:

- Dashboard;
- Cadastro;
- Tabela de Itens;

- Materiais;
- Todos;
- Ativos;
- Inativos.

Esses botões facilitam a navegação entre as seções e a visualização dos materiais conforme o status.

---

## 12. Organização do código

O projeto foi organizado em pastas e arquivos separados para facilitar a manutenção e a leitura do código.

---

### Pasta `Models`

A pasta `Models` contém as classes que representam as entidades do sistema.

#### `MaterialEscolar.cs`

Arquivo responsável por definir a estrutura da entidade `MaterialEscolar`.

Contém os campos:

- `Id`
- `Nome`
- `Categoria`
- `Ativo`
- `Lotes`

Essa classe representa o material principal controlado pela API.

---

#### `Lote.cs`

Arquivo responsável por definir a estrutura da entidade `Lote`.

Contém os campos:

- `Id`
- `Codigo`
- `Quantidade`
- `DataEntrada`
- `DataVencimento`
- `ValorCusto`
- `ValorVenda`

Essa classe representa os lotes vinculados aos materiais.

---

### SaidaEstoque.cs

Arquivo responsável por definir a estrutura da requisição de saída de estoque.

Contém os campos:

- MaterialId
  - Quantidade
- 

## Pasta Data

### BancoSimulado.cs

Arquivo responsável por armazenar os materiais simulados em memória.

O arquivo contém uma lista de materiais escolares, seus lotes, quantidades, valores e datas.

Na versão atual, a base simulada pode gerar uma quantidade maior de materiais para demonstração, permitindo testar o dashboard, os filtros e a tabela de itens.

---

## Pasta Routes

A pasta Routes contém os arquivos responsáveis pelas rotas da API. Cada arquivo possui uma responsabilidade específica.

### ROTA\_GET.cs

Responsável pelas rotas de consulta geral dos materiais.

Contém:

- Rota raiz /;
  - Consulta de todos os materiais /api/materiais;
  - Consulta por ID /api/materiais/{id};
  - Consulta de lotes /api/materiais/{id}/lotos.
- 

### ROTA\_GET\_ATIVOS.cs

Responsável pela rota que retorna apenas os materiais ativos.

Rota implementada:

```
GET /api/materiais/ativos
```

---

#### ROTA\_GET\_INATIVOS.cs

Responsável pela rota que retorna apenas os materiais inativos.

Rota implementada:

```
GET /api/materiais/inativos
```

---

#### ROTA\_POST.cs

Responsável pela rota de cadastro de novos materiais.

Rota implementada:

```
POST /api/materiais
```

O material recebido no corpo da requisição é adicionado à lista simulada.

---

#### ROTA\_PUT.cs

Responsável pela rota de edição de materiais.

Rota implementada:

```
PUT /api/materiais/{id}
```

A rota localiza o material pelo `Id` e atualiza os dados enviados no corpo da requisição.

---

#### ROTA\_DELETE.cs

Responsável pela rota de exclusão de materiais.

Rota implementada:

```
DELETE /api/materiais/{id}
```

A rota localiza o material pelo `Id` e remove da lista caso ele exista.

---

### ROTA\_SAIDA.cs

Responsável pela rota de saída de estoque.

Rota implementada:

```
POST /api/materiais/saida
```

Essa rota valida a quantidade solicitada, verifica se há estoque suficiente e reduz a quantidade dos lotes disponíveis.

---

### ROTA\_DASHBOARD.cs

Responsável pela rota do dashboard.

Rota implementada:

```
GET /api/dashboard
```

Essa rota calcula e retorna os indicadores operacionais e financeiros do sistema.

---

## Program.cs

O arquivo `Program.cs` é o ponto de entrada da aplicação.

Ele é responsável por:

- Criar o builder da aplicação;
- Configurar o Swagger;
- Configurar o CORS;
- Criar a aplicação;
- Ativar o Swagger e a interface Swagger UI;
- Registrar todas as rotas criadas na pasta `Routes`;
- Inicializar a execução da API.

No arquivo, as rotas são registradas da seguinte forma:

```
app.MapGetRoutes();  
app.MapGetAtivosRoutes();  
app.MapGetInativosRoutes();  
app.MapPostRoutes();  
app.MapPutRoutes();  
app.MapDeleteRoutes();
```

```
app.MapSaidaRoutes();  
app.MapDashboardRoutes();
```

---

## Pasta `frontend`

A pasta `frontend` contém a interface web do sistema.

`index.html`

Arquivo responsável pela estrutura da tela.

Contém as seções:

- Dashboard;
  - Dashboard financeiro por material;
  - Cadastro;
  - Saída de estoque;
  - Tabela de itens, códigos e lotes;
  - Lista de materiais cadastrados.
- 

`style.css`

Arquivo responsável pela estilização da interface.

Define:

- Modo dark;
  - Layout minimalista;
  - Cards;
  - Tabelas;
  - Botões;
  - Formulários;
  - Responsividade;
  - Cores de status;
  - Estilos dos botões de editar e remover.
- 

`script.js`

Arquivo responsável pela comunicação entre o front-end e a API.

Utiliza `fetch` para consumir as rotas:

- `GET /api/materiais;`
- `GET /api/materiais/ativos;`
- `GET /api/materiais/inativos;`

- `POST /api/materiais;`
- `PUT /api/materiais/{id};`
- `DELETE /api/materiais/{id};`
- `POST /api/materiais/saida;`
- `GET /api/dashboard.`

Também é responsável por atualizar a tela, renderizar as tabelas, preencher o formulário de edição, calcular informações financeiras no front e controlar a navegação entre seções.

---

## 13. Arquivos de configuração

`appsettings.json`

Arquivo utilizado para armazenar configurações gerais da aplicação.

`appsettings.Development.json`

Arquivo utilizado para configurações específicas do ambiente de desenvolvimento.

`launchSettings.json`

Arquivo localizado na pasta `Properties`, responsável por armazenar configurações de inicialização do projeto durante o desenvolvimento.

---

## 14. Como executar o projeto

### Back-end

No terminal, acesse a pasta do projeto onde está o arquivo `.csproj`:

```
cd C:\Users\emily.garcia\WMS_API\WAREHOUSESYSTEM_BACKEND\ApiProdutos
```

Restaure as dependências:

```
dotnet restore
```

Execute a API:

```
dotnet run
```

A API será iniciada em uma URL semelhante a:



```
http://localhost:5181
```

O Swagger pode ser acessado em:

```
http://localhost:5181/swagger
```

---

## Front-end

Acesse a pasta `frontend` no VS Code.

Abra o arquivo `index.html` com a extensão **Live Server**.

A URL será semelhante a:

```
http://127.0.0.1:5500/frontend/index.html
```

No arquivo `script.js`, a URL da API deve estar configurada corretamente:

```
const API_URL = "http://localhost:5181";
```

Caso a API execute em outra porta, é necessário atualizar essa constante.

---

## 15. Justificativa técnica

A equipe optou por desenvolver a API utilizando **ASP.NET Core Minimal API**, pois essa abordagem permite criar endpoints de forma simples, rápida e objetiva, sem a necessidade de estruturas mais complexas como controllers tradicionais.

A separação das rotas em arquivos diferentes também foi uma decisão técnica importante, pois melhora a organização do projeto e facilita a manutenção do código. Dessa forma, cada arquivo possui uma responsabilidade específica, como consulta, cadastro, edição, exclusão, saída de estoque e dashboard.

Outra decisão adotada foi a utilização de uma lista simulada em memória para armazenar os dados. Essa escolha foi feita por se tratar de um projeto acadêmico, com foco principal na criação e funcionamento dos endpoints da API. Em um ambiente real, essa estrutura poderia ser evoluída para utilizar um banco de dados, como SQL Server, PostgreSQL, MySQL ou SQLite.

Também foi implementado o Swagger, que facilita a visualização, teste e documentação dos endpoints da API. Com ele, é possível testar as rotas diretamente pelo navegador, tornando o desenvolvimento e a apresentação do projeto mais simples.

O front-end foi desenvolvido com HTML, CSS e JavaScript puro, permitindo consumir os endpoints da API de forma direta com `fetch`. A interface em modo dark foi escolhida para melhorar a visualização e trazer uma aparência mais moderna ao sistema.

A criação das entidades `MaterialEscolar`, `Lote` e `SaidaEstoque` permite representar uma estrutura próxima de um sistema real de estoque, onde cada material pode possuir diferentes lotes, quantidades, datas de entrada, datas de vencimento e valores financeiros.

A implementação da saída por lote torna a solução mais próxima de um WMS real, pois considera regras de controle de estoque como a priorização de lotes com vencimento mais próximo.

A inclusão do dashboard financeiro amplia a proposta do sistema, permitindo que o usuário visualize não apenas a quantidade de itens em estoque, mas também indicadores como valor investido, venda estimada, lucro estimado e ROI.

---

## 16. Possíveis melhorias futuras

Como evolução do projeto, poderiam ser implementadas as seguintes melhorias:

- Integração com banco de dados;
- Histórico de movimentações de entrada e saída;
- Cadastro de múltiplos lotes por material;
- Edição individual de lotes;
- Controle de usuários;
- Login e autenticação;
- Níveis de permissão;
- Alertas de estoque baixo;
- Alertas de vencimento próximo;
- Relatórios exportáveis;
- Dashboard com gráficos;
- Integração com Power BI;
- Melhorias de acessibilidade;
- Responsividade avançada;
- Deploy da API e do front-end.